Toward Verifiable Performance Engineering

High-performance programming is often treated as an art: expertise accumulates through experience, and performance improves through repeated implementation and measurement. An optimized implementation records the decisions an expert made, but not why they were necessary, so that reasoning must be rebuilt for each new application and machine. My research asks how performance engineering can instead become a systematic discipline.

I study performance patterns: recurring mappings between the functional dataflow of an application and the structure of the hardware that executes it. The specific realization changes, but the relationship between a hardware bottleneck and an appropriate design response often does not. A decision to preserve reuse, for example, may begin as an application-level dataflow choice and later constrain operator composition, loop organization, and register use. My goal is to identify these relationships, build abstractions that preserve them during refinement, and derive performance contracts that evaluate whether a realization uses the target hardware well and explain why when it does not.

Performance patterns

My dissertation work on SMaLL showed that performance expertise can be encoded in reusable implementation structure. SMaLL organizes computation around shared microkernel interfaces and orchestration loops, extending the approach BLIS established for linear algebra to the structurally messier domain of CNN operators. Working alone, I brought the framework up on six edge platforms, from multicore x86 to ARM microcontrollers, extended it to quantized data types in a day, and covered a large fraction of ONNX operators without writing hardware-specialized code for each. The lesson was not that operators could be made fast but that they had to compose: SMaLL shares one data layout across all operators, because a locally efficient implementation can still perform poorly if its output representation forces unnecessary work downstream.

Algorithmic categories alone are therefore the wrong unit of performance reasoning, because nominally different computations can run into the same hardware constraint. What matters is the relationship between a computation's dataflow and the resources available to realize it: a bandwidth-limited computation may need a design that preserves locality rather than one that reduces arithmetic work. I study these relationships using analytical hardware models, which describe whether a proposed mapping is aligned with a model of the target hardware rather than predicting its runtime. My near-term work will apply this to FFTs and set-intersection kernels, which stress hardware differently.

Abstractions for refinement

My position is that good performance comes from a hardware-aligned design: once that design is

known, fast code can be written quickly in a low-level language or in a domain-specific abstraction. A new abstraction or language for performance is therefore only as useful as the designs it can express, and performance patterns are that content. Many abstractions are instead built to make expression cheap so that search can find a fast candidate; search is effective, but the result records what worked, not why.

I evaluate abstractions along two axes. Expressivity asks whether an abstraction can state the designs we care about, which a corpus of performance patterns makes testable. Alignment asks how closely its primitives correspond to the machine beneath them, and therefore how directly a hardware-aligned design can be stated, much as linguists measure how codable a concept is in a language. SMaLL absorbing quantized data types in a day is an alignment result, not only a portability one.

The obstacle is that interfaces fix what crosses them. BLAS packs two-dimensional matrices internally, and a caller who would accept a blocked result has no way to say so; at the sizes attention produces, that packing is a substantial fraction of runtime. Likewise, a vector-width exp cannot see that safe-softmax guarantees non-positive inputs, or that a tensor tile offers enough independent work to hide its latency. I plan to express SMaLL's patterns in scheduling systems such as Exo and Triton and ask whether their abstractions retain the information needed to state the original design, distinguishing a poor scheduling decision from an abstraction that can no longer represent an important constraint. An undergraduate can own a piece of this, such as exposing exp as a tensor-level primitive with context-specific variants.

Designing performant implementations to a spec

Correctness is checked against a specification; performance usually isn't, because a spec that states only what a program computes leaves out half of what an implementation must satisfy. My third thrust asks whether the conditions for good performance can be stated and checked in advance. In Gen-AI in a Bottle, I showed that encoding expert design knowledge into prompts lets a code LLM generate GEMM microkernels, the innermost matrix-multiply routines that dominate ML workloads, reaching 99% of peak throughput for single and double precision on x86 and ARM targets. The encoded design was a sequence of transformations, from loop order to register allocation, each leaving a signature in the generated code that can be checked without running it; when a candidate falls short, the missing transformation is what to report.

I want to derive performance contracts from three inputs: a selected performance pattern, its desired instantiation parameters such as problem size and data type, and the specifics of the target hardware. Whether a pattern applies to an application is checked separately against its dataflow; the contract instantiates the pattern against the hardware's constraints and establishes whether it can satisfy them. If it cannot, another pattern is needed; if it can, the contract specifies what an

implementation must exhibit and, when a candidate falls short, which design decisions to revisit. A contract does not predict runtime; it checks whether the way an implementation expresses its work can reach the performance ceiling set by the modeled hardware bottlenecks.

My dissertation provides a basis for such contracts at the scale of a single microkernel. For the outer-product pattern, which expresses matrix multiplication as a sum of outer products, an implementation must expose enough independent operations to keep every pipeline stage busy while holding its operands and accumulators within the available registers. Since each independent operation consumes a register, these constraints compete, and checking whether both can be met determines whether the pattern is feasible. The hardest open problem is composition: SMaLL guarantees compatible layouts by construction, but it is unclear how to state such conditions so that kernel contracts combine.

Because contracts exist before complete code does, they can guide implementation rather than only judge it. A current project uses them as feedback for an agent that designs microkernels, checking before anything is compiled whether its choices keep data in fast memory and exercise the intended datapath. Correctness is the complementary side. In ongoing work, I am using Lean to develop formally verified orchestration loops that compose specialized kernels. I want to know whether the abstractions that make specialization manageable also make correctness modularly provable.

Together, these thrusts aim to change what an optimized implementation records. Performance patterns capture why a design is necessary; abstractions that preserve them let that reasoning survive refinement onto new hardware; and contracts make it checkable by people, by compilers, and by AI systems that increasingly write low-level code. The result is performance engineering that transfers: when a new architecture or application arrives, the reasoning is restated against a new dataflow or hardware model rather than rebuilt by hand.